Aula03 – Banco de Dados

Regras de Nomenclatura

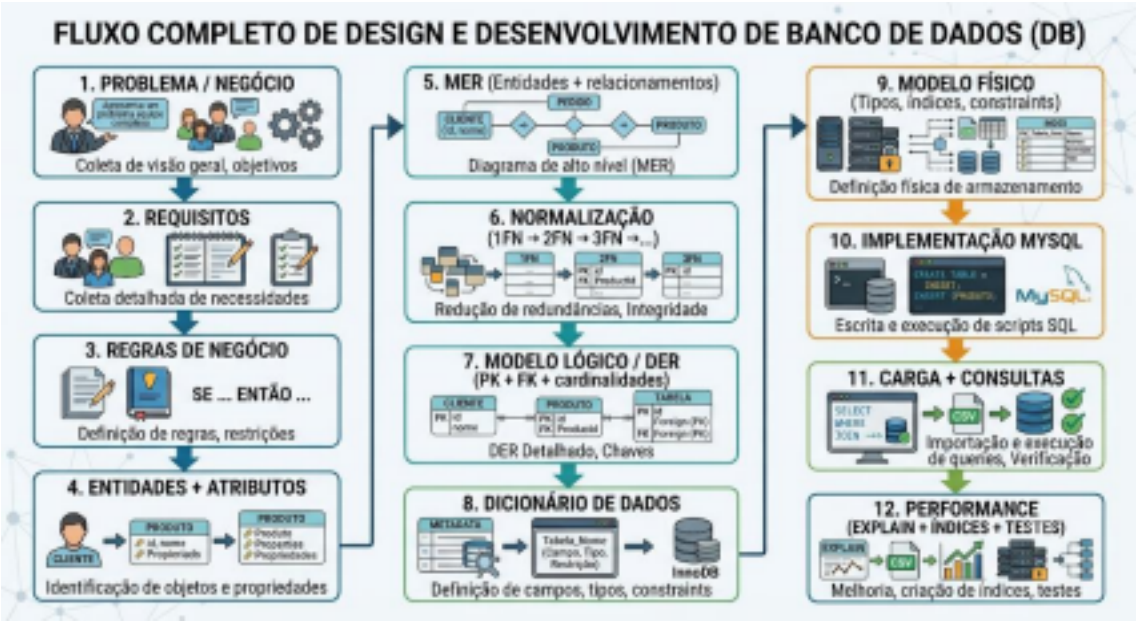
Singular ou Plural?

- Prefira o **plural** (ex: clientes, produtos, pedidos).
- Cada linha da tabela é um item. A tabela guarda o grupo todo. • O plural combina bem com a ideia de uma coleção de dados.

Maiúscula ou Minúscula?

- Use letras **minúsculas**.
- Evite letras maiúsculas para não gerar conflitos entre sistemas operacionais (como Windows e Linux).
- Use o formato snake_case (letras minúsculas separadas por sublinhado) se o nome tiver duas palavras (ex: pedido_itens).

Fluxo de Design e Desenvolvimento de Banco de Dados





Exemplo do Modelo Entidade-Relacionamento (MER)

Esta estrutura mínima é o padrão para sistemas de vendas e e-commerce.
Entidades:

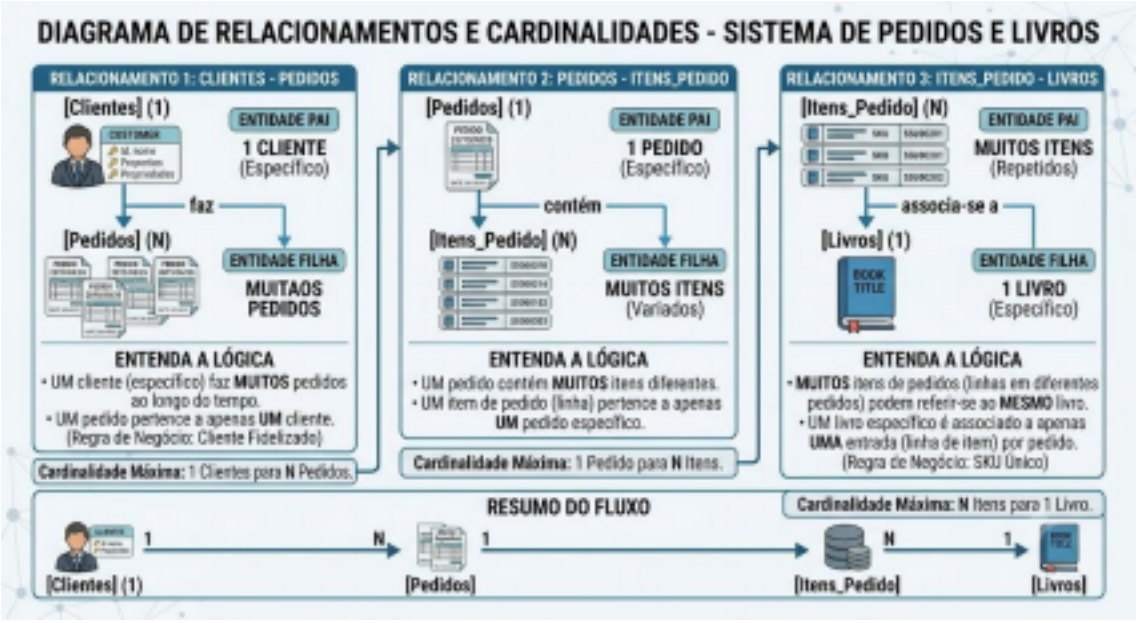
- clientes;
- pedidos;
- itens_pedido;
- livros.

Elementos do Exemplo (Entidades e Atributos)

- **Clientes** (Entidade)
  - idCliente (Chave Primária - PK)
  - nome
  - email
- **Pedidos** (Entidade)
  - idPedido (Chave Primária - PK)
  - idCliente (Chave Estrangeira - FK, aponta para Clientes)
  - dataPedido
  - valorTotal
- **Itens_Pedido** (Entidade Associativa)
  - idPedido (Chave Primária Composta / Chave Estrangeira - PK/FK)
  - idLivro (Chave Primária Composta / Chave Estrangeira - PK/FK)
  - quantidade

- precoUnitarioVenda (Preço cobrado no momento da compra) • **Livros** (Entidade)
  - idLivro (Chave Primária - PK)
  - titulo
  - precoAtual (Preço atual no catálogo)

Relacionamentos e Cardinalidades





- **Clientes ➔ Pedidos (1:N)**
  - **Regra:** Um cliente pode realizar **vários** pedidos ao longo do tempo. Um pedido pertence obrigatoriamente a **um único** cliente.
- **Pedidos ➔ Itens_Pedido (1:N)**
  - **Regra:** Um pedido contém **um ou vários** itens (linhas do carrinho). Cada registro de item pertence a **um único** pedido.

- **Livros ➔ Itens_Pedido (1:N)**
    - **Regra:** Um livro pode ser incluído em **vários** itens de pedidos de clientes diferentes. Cada registro de item faz referência a **um único** livro específico.

Simulação dos dados e o script SQL correspondente para você visualizar e testar o seu modelo na prática.

Simulação com Dados Fictícios

Repare em como as chaves (id) conectam as tabelas.

O cliente **Carlos** faz um pedido, e esse pedido contém **dois livros diferentes** mapeados na tabela Itens_Pedido.

Tabela: Clientes

idCliente (PK)	nome	email
1	Carlos Silva	carlos@email.com
2	Ana Souza	ana@email.com

Tabela: Pedidos

idPedido (PK)	idCliente (FK)	dataPedido	valorTotal
1001	**1** *(Carlos)*	2026-08-10	150.00

Tabela: Livros

idLivro (PK)	titulo	precoAtual
50	Aprendendo SQL	90.00
60	Modelagem de Dados	60.00

Tabela: Itens_Pedido

idPedido (PK, FK)	idLivro (PK, FK)	quantidade	precoUnitarioVenda
1001	**50** *(SQL)*	1	90.00

1001	**60** *(Modelagem)*	1	60.00

2. Script SQL de Criação (DDL)

Este script cria as tabelas respeitando a ordem correta de dependência (as tabelas que possuem Chaves Estrangeiras são criadas por último).

```sql
-- 1. Cria e seleciona o banco de dados
CREATE DATABASE IF NOT EXISTS ecommerce_db;
USE ecommerce_db;

-- 2. Criação da tabela de Clientes
CREATE TABLE IF NOT EXISTS clientes (
    id_cliente INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    cpf VARCHAR(11) NOT NULL UNIQUE,
    telefone VARCHAR(15),
    data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Criação da tabela de Produtos
CREATE TABLE IF NOT EXISTS produtos (
    id_produto INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    descricao TEXT,
    preco DECIMAL(10, 2) NOT NULL,
    estoque INT NOT NULL DEFAULT 0,
    data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Criação da tabela de Pedidos
CREATE TABLE IF NOT EXISTS pedidos (
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,
    id_cliente INT NOT NULL,
    data_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50) DEFAULT 'Pendente',
    total DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
    FOREIGN KEY (id_cliente) REFERENCES clientes(id_cliente) ON DELETE RESTRICT
);

-- 5. Criação da tabela de Itens do Pedido (Relacionamento Muitos para Muitos)
CREATE TABLE IF NOT EXISTS itens_pedidos (
    id_item INT AUTO_INCREMENT PRIMARY KEY,
    id_pedido INT NOT NULL,
    id_produto INT NOT NULL,
    quantidade INT NOT NULL,
    preco_unitario DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (id_pedido) REFERENCES pedidos(id_pedido) ON DELETE CASCADE,
    FOREIGN KEY (id_produto) REFERENCES produtos(id_produto) ON DELETE RESTRICT
);
```



Pontos importantes da estrutura:

A cláusula IF NOT EXISTS:

Se o banco de dados já existir, **nada acontece** e nenhum erro é retornado.

A cláusula **IF NOT EXISTS** serve justamente como uma proteção para o seu script SQL.

O Comportamento Detalhado

- **Sem a cláusula** (CREATE DATABASE ecommerce_db): O banco de dados tenta ser criado. Se ele já existir, o sistema interrompe a execução e retorna um erro (ex: *Database 'ecommerce_db' already exists*).
- **Com a cláusula** (CREATE DATABASE IF NOT EXISTS ecommerce_db): O sistema verifica a existência do banco primeiro. Se ele já estiver lá, o banco de dados atual **não é alterado, apagado ou substituído**. O comando é simplesmente ignorado com sucesso, exibindo apenas um aviso (*warning*), permitindo que o restante do seu script continue rodando sem interrupções.
- **DECIMAL(10,2):** Usado para preços. Evita problemas de arredondamento que ocorrem com o tipo FLOAT.
- **UNIQUE:** Garante que não existam dois clientes com o mesmo e-mail ou CPF.
- **ON DELETE RESTRICT:** Impede que um produto ou cliente seja apagado se ele já estiver vinculado a um pedido real.
- **ON DELETE CASCADE:** Se um pedido for excluído, todos os itens associados a ele na tabela itens_pedidos também serão apagados automaticamente.

3. Script SQL de Inserção (DML)

Para testar no seu banco de dados, você pode rodar este bloco que insere os mesmos dados da simulação acima:

sql

```sql
USE ecommerce_db;

-- ============================================================================
-- 1. INSERÇÃO DE 5 CLIENTES
-- ============================================================================
INSERT INTO clientes (id_cliente, nome, email, cpf, telefone) VALUES
(1, 'Ana Souza', 'ana.souza@email.com', '11122233344', '11911112222'),
(2, 'Bruno Lima', 'bruno.lima@email.com', '22233344455', '11922223333'),
(3, 'Carlos Gomes', 'carlos.gomes@email.com', '33344455566', '11933334444'),
(4, 'Daniela Reis', 'daniela.reis@email.com', '44455566677', '11944445555'),
(5, 'Eduardo Costa', 'eduardo.costa@email.com', '55566677788', '11955556666');

-- ============================================================================
-- 2. INSERÇÃO DE 10 LIVROS (PRODUTOS)
-- ============================================================================
INSERT INTO produtos (id_produto, nome, descricao, preco, estoque) VALUES
(1, 'Código Limpo', 'Habilidades Práticas do Software Agile', 85.00, 50),
(2, 'Arquitetura Limpa', 'O Guia do Artesão para Estrutura', 92.50, 40),
(3, 'O Programador Pragmático', 'Sua Jornada Para A Maestria', 110.00, 35),
(4, 'Padrões de Projetos', 'Soluções Reutilizáveis de Software', 145.00, 25),
(5, 'Refatoração', 'Aperfeiçoando o Design de Códigos', 120.00, 30),
(6, 'SQL e Modelagem de Dados', 'Guia Prático de Banco de Dados', 65.00, 60),
(7, 'Entendendo Algoritmos', 'Um Guia Ilustrado Para Programadores', 59.90, 70),
(8, 'Domain-Driven Design', 'Atacando a Complexidade no Coração', 155.00, 20),
(9, 'Introdução ao Concurrency', 'Programação Concorrente na Prática', 78.00, 45
(10, 'Git Completo', 'Controle de Versão do Iniciante ao Avançado', 42.00, 80);
```



```sql
-- ============================================================================
-- 3. INSERÇÃO DE 20 PEDIDOS (MATEMÁTICA INTEGRADA COM OS ITENS)
-- ============================================================================
INSERT INTO pedidos (id_pedido, id_cliente, status, total) VALUES
(1, 1, 'Entregue', 287.50),  -- Itens: Livros 1, 2, 3
(2, 2, 'Pago', 385.00),      -- Itens: Livros 4, 5, 6
(3, 3, 'Pendente', 292.90),  -- Itens: Livros 7, 8, 9
(4, 4, 'Entregue', 212.00),  -- Itens: Livros 10, 1, 2
(5, 5, 'Pago', 375.00),      -- Itens: Livros 3, 4, 5
(6, 1, 'Pendente', 279.90),  -- Itens: Livros 6, 7, 8
(7, 2, 'Entregue', 205.00),  -- Itens: Livros 9, 10, 1
(8, 3, 'Pago', 347.50),      -- Itens: Livros 2, 3, 4
(9, 4, 'Pendente', 244.90),  -- Itens: Livros 5, 6, 7
(10, 5, 'Entregue', 388.00), -- Itens: Livros 8, 9, 10
(11, 1, 'Pago', 287.50),     -- Itens: Livros 1, 2, 3
(12, 2, 'Pendente', 385.00), -- Itens: Livros 4, 5, 6
(13, 3, 'Entregue', 292.90), -- Itens: Livros 7, 8, 9
(14, 4, 'Pago', 212.00),     -- Itens: Livros 10, 1, 2
(15, 5, 'Pendente', 375.00), -- Itens: Livros 3, 4, 5
(16, 1, 'Entregue', 279.90), -- Itens: Livros 6, 7, 8
(17, 2, 'Pago', 205.00),     -- Itens: Livros 9, 10, 1
(18, 3, 'Pendente', 347.50), -- Itens: Livros 2, 3, 4
(19, 4, 'Entregue', 244.90), -- Itens: Livros 5, 6, 7
(20, 5, 'Pago', 388.00);     -- Itens: Livros 8, 9, 10
```



```sql
-- ============================================================================
-- 4. INSERÇÃO DE 60 ITENS DE PEDIDO (EXATAMENTE 3 ITENS POR PEDIDO)
-- ============================================================================
INSERT INTO itens_pedidos (id_pedido, id_produto, quantidade, preco_unitario) VA
-- Pedido 1
(1, 1, 1, 85.00), (1, 2, 1, 92.50), (1, 3, 1, 110.00),
-- Pedido 2
(2, 4, 1, 145.00), (2, 5, 1, 120.00), (2, 6, 1, 65.00),
-- Pedido 3
(3, 7, 1, 59.90), (3, 8, 1, 155.00), (3, 9, 1, 78.00),
-- Pedido 4
(4, 10, 1, 42.00), (4, 1, 1, 85.00), (4, 2, 1, 92.50),
-- Pedido 5
(5, 3, 1, 110.00), (5, 4, 1, 145.00), (5, 5, 1, 120.00),
-- Pedido 6
(6, 6, 1, 65.00), (6, 7, 1, 59.90), (6, 8, 1, 155.00),
-- Pedido 7
(7, 9, 1, 78.00), (7, 10, 1, 42.00), (7, 1, 1, 85.00),
-- Pedido 8
(8, 2, 1, 92.50), (8, 3, 1, 110.00), (8, 4, 1, 145.00),
-- Pedido 9
(9, 5, 1, 120.00), (9, 6, 1, 65.00), (9, 7, 1, 59.90),
-- Pedido 10
(10, 8, 1, 155.00), (10, 9, 1, 78.00), (10, 10, 1, 42.00),
-- Pedido 11
(11, 1, 1, 85.00), (11, 2, 1, 92.50), (11, 3, 1, 110.00),
-- Pedido 12
(12, 4, 1, 145.00), (12, 5, 1, 120.00), (12, 6, 1, 65.00),
-- Pedido 13
(13, 7, 1, 59.90), (13, 8, 1, 155.00), (13, 9, 1, 78.00),
-- Pedido 14
(14, 10, 1, 42.00), (14, 1, 1, 85.00), (14, 2, 1, 92.50),
-- Pedido 15
(15, 3, 1, 110.00), (15, 4, 1, 145.00), (15, 5, 1, 120.00),
-- Pedido 16
(16, 6, 1, 65.00), (16, 7, 1, 59.90), (16, 8, 1, 155.00),
-- Pedido 17
(17, 9, 1, 78.00), (17, 10, 1, 42.00), (17, 1, 1, 85.00),
-- Pedido 18
```

